

MAHALO Notebook 1

What the DC34 Captures Reveal

Mesh Assessment of Heard And Lost Offerings

by Abraxas3d for Open Research Institute, Inc.

Sending Stones project <https://github.com/OpenResearchInstitute/sending-stones>

Open Research Institute <https://openresearch.institute>

Graciously enabled by Vid through RF Village Discord (thank you!)

Inputs (local only, never republished): per contributor request, the datasets analyzed here are not distributed with this notebook. Paths below point at local copies. This notebook renders only aggregates. Counts, histograms, and time series. Node identifiers appear only as anonymous rank labels (S1, S2, ...). Sender names and message payloads are never selected into memory. What this accomplishes is that personally-identifiable informatin (PII) is kept private by construction.

Node identifiers are rendered as rank labels (S1–S10) rather than raw device IDs, to avoid attributing specific behavior to identifiable devices. Collector handles and capture locations are retained as contributor credits.

Provenance (to be completed as confirmed):

Capture	Collector	Position	Instrument	Coverage
a5fe CSV	unknown (node ... a5fe)	unknown (con floor?)	app/serial datalog, deduplicated by packet	Aug 5–11
feeb-fb31.db	"Feeb"	Fontainebleau, 31st floor	megalogger (device trace tail → SQLite), duplicates preserved	Aug 3–9

Headline caution, pre-registered: none of these captures contains a delivery ratio. No offered load was controlled, so no denominator exists. Everything here characterizes what was *heard* at specific apertures and not what was *delivered*. Sending Stones controls an offered load, therefore the experiment will produce a denominator.

```
In [1]: # ---- Parameters: edit paths for your machine ----
A5FE_CSV = "Meshtastic_datalog_a5fe_20260811_144102.csv" # Blue-Palace: ~/M
FEEB_DB  = "feeb-fb31.db"                               # Blue-Palace: ~/M

STORM_THRESHOLD = 15 # copies; > N defines the pathological ("storm") clas
```

```

import os, sqlite3
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
plt.rcParams.update({"figure.figsize": (9, 3.2), "figure.dpi": 110})
HAVE_FEEB = os.path.exists(FEEB_DB)
print(f"a5fe: {os.path.exists(A5FE_CSV)}    feeb: {HAVE_FEEB}")

```

a5fe: True feeb: True

1 Aperture A: the a5fe app-level datalog

A single node's log, app-deduplicated (one row per unique packet heard). Columns kept: timestamps, numeric sender id, port tag, hop fields, SNR. **Not** loaded: sender names, payload text.

```

In [2]: # What columns are we going to use
usecols = ["date", "time", "from", "rx snr", "hop limit", "hop start", "payload"]
a = pd.read_csv(A5FE_CSV, usecols=usecols)
a["ts"] = pd.to_datetime(a["date"] + " " + a["time"])
# port tag only; drop payload text immediately
a["port"] = a["payload"].str.extract(r"^( [A-Z_]+>")[0].fillna("TEXT/other")
a = a.drop(columns=["payload", "date", "time"])
print(f"rows={len(a)}    span={a.ts.min()} to {a.ts.max()}    unique senders={a

```

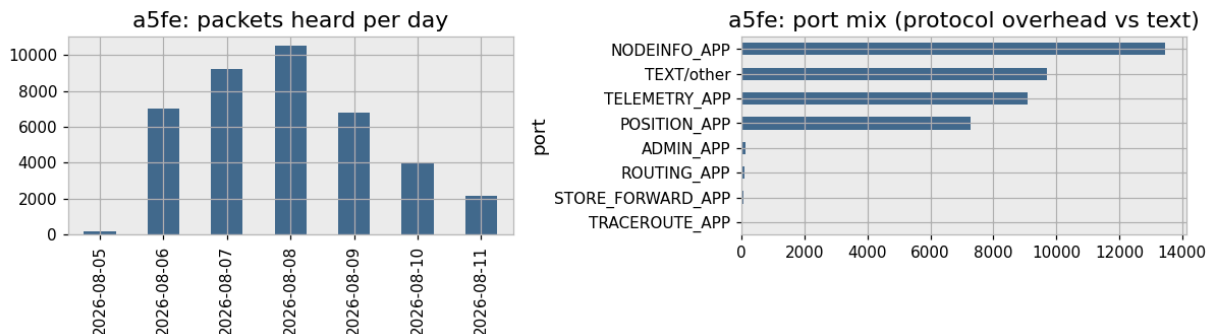
rows=39837 span=2026-08-05 22:50:33 to 2026-08-11 14:41:06 unique senders=2123

```

In [3]: fig, ax = plt.subplots(1, 2, figsize=(11, 3.2))
a.groupby(a.ts.dt.date).size().plot(kind="bar", ax=ax[0], color="#456990")
ax[0].set_title("a5fe: packets heard per day"); ax[0].set_xlabel("")
a["port"].value_counts().plot(kind="barh", ax=ax[1], color="#456990")
ax[1].set_title("a5fe: port mix (protocol overhead vs text)")
ax[1].invert_yaxis(); plt.tight_layout()
ov = a["port"].isin(["NODEINFO_APP", "TELEMETRY_APP", "POSITION_APP", "ADMIN_AF
print(f"protocol housekeeping share: {ov:.0%}")

```

protocol housekeeping share: 75%



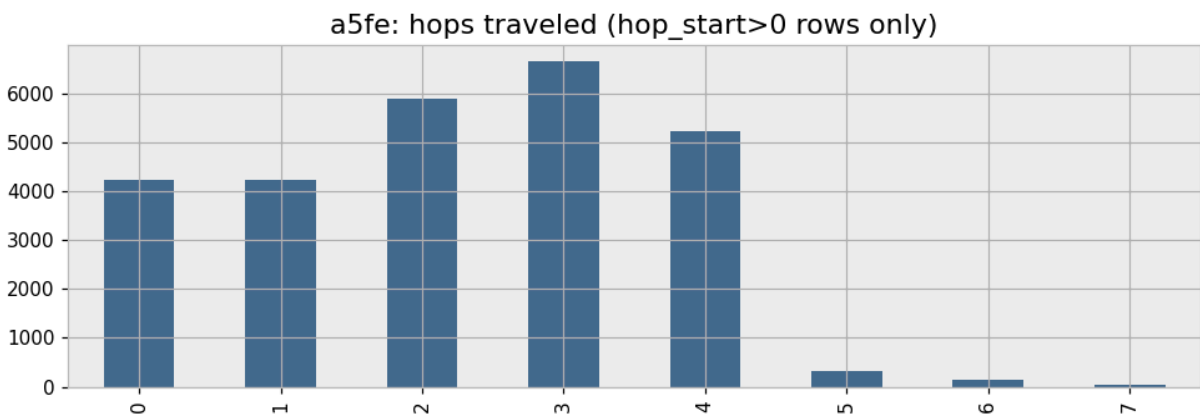
Parser doctrine #1 (learned here): rows where `hop start == 0` mean *unknown hops* (older firmware doesn't populate the field), and not zero hops. Computing `hop_start - hop_limit` without that filter manufactures a phantom “-4 hops” population. Ask me how I know :D

```
In [4]: h = a[a["hop start"] > 0]
hops = (h["hop start"] - h["hop limit"])
axh = hops.value_counts().sort_index().plot(kind="bar", color="#456990")
axh.set_title("a5fe: hops traveled (hop_start>0 rows only)"); plt.tight_layout()
print(f"known-hop rows: {len(h)} of {len(a)}")
print(f">=4-hop share of known: {(hops>=4).mean():.1%}")
print(f"event firmware capped 3, so 4+ is default-firmware flood behavior")
```

known-hop rows: 26736 of 39837

>=4-hop share of known: 21.4%

event firmware capped 3, so 4+ is default-firmware flood behavior



2 Aperture B: feeb-fb31.db (Fontainebleau 31st floor, duplicates preserved)

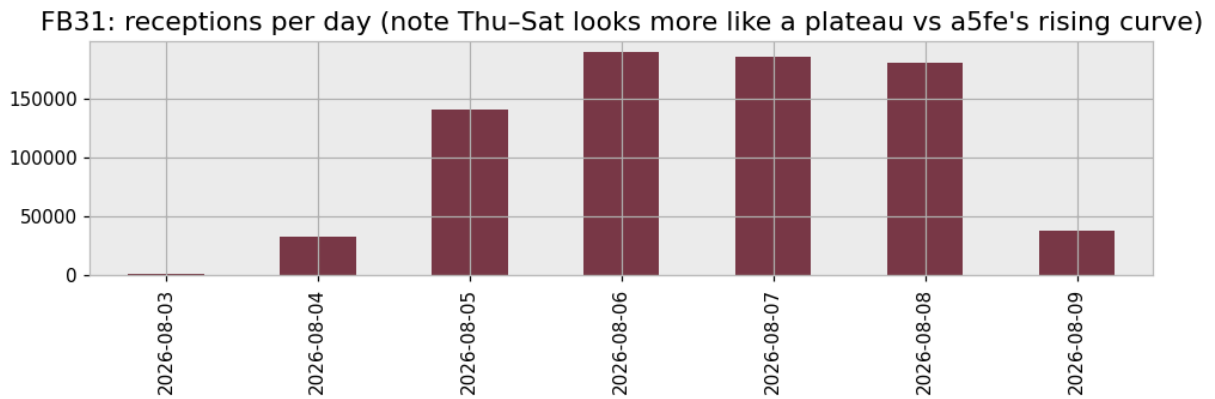
This is a tailing logger with resume cursors and per-line SHA-256. Crucially, every RF reception kept, including encrypted traffic (headers only decode). ~97% of receptions are header-only. They are countable and joinable but unreadable. This cleanly separates flood physics from message privacy. All cells below no-op gracefully if the database path is absent.

```
In [5]: def q(sql):
    if not HAVE_FEEB:
        print("feeb db not present; skipping"); return pd.DataFrame()
    with sqlite3.connect(FEEB_DB) as cx:
        return pd.read_sql_query(sql, cx)

counts = q("""SELECT 'records' k, COUNT(*) n FROM trace_records
UNION ALL SELECT 'raw', COUNT(*) FROM trace_raw_packets
UNION ALL SELECT 'decoded', COUNT(*) FROM trace_decoded_packets""")
census = q("SELECT COUNT(DISTINCT from_node) n FROM trace_raw_packets")
print(counts.to_string(index=False))
if HAVE_FEEB: print(f"unique senders heard: {census.n[0]}")
```

k	n
records	814441
raw	789041
decoded	25400
unique senders heard:	2782

```
In [6]: daily = q("""SELECT date(trace_timestamp,'unixepoch') d, COUNT(*) n
FROM trace_raw_packets WHERE trace_timestamp > 1e9 GROUP BY 1 ORDER BY 1""")
if len(daily):
    daily.plot(x="d", y="n", kind="bar", legend=False, color="#7a3b46")
    plt.title("FB31: receptions per day (note Thu-Sat looks more like a plateau vs a5fe's rising curve)
    plt.xlabel(""); plt.tight_layout()
```

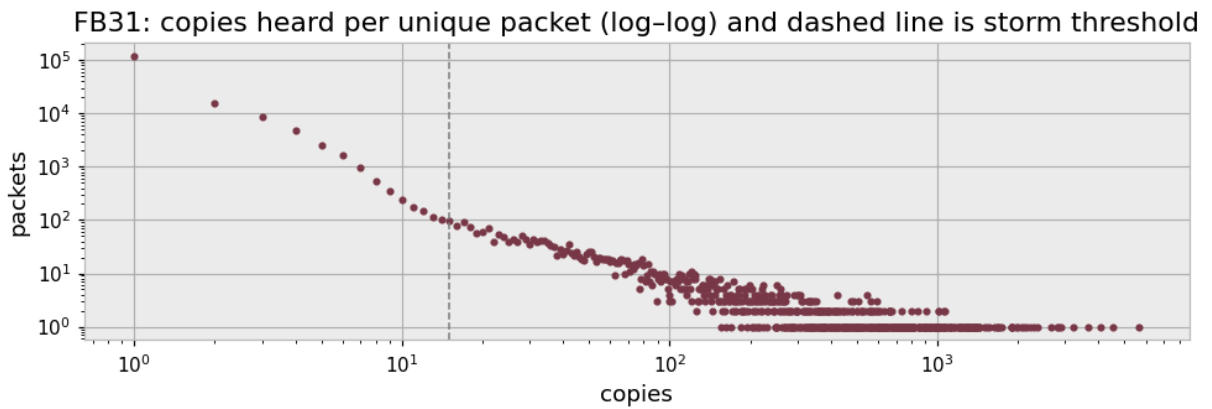


2.1 Flood duplication & the storm class

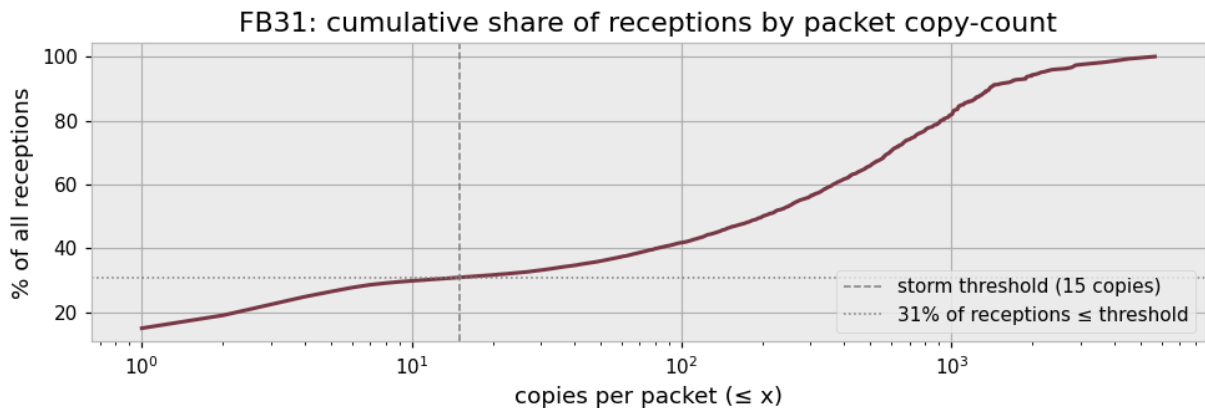
(from_node, packet_id) is a global packet identity. Copies-per-packet measures flood fan-out at this aperture. The hop_start == hops_away signature marks receptions arriving with full budget. I claim this means that they were **heard directly from an originator**. Thousands of full-budget copies of a single packet id mean the same packet was re-originated at machine rate (retry storm and/or MQTT gateway re-injection), and not merely relayed.

```
In [7]: cp = q("""SELECT n_copies, COUNT(*) n_packets FROM
(SELECT from_node, packet_id, COUNT(*) n_copies FROM trace_raw_packets GROUP
GROUP BY 1 ORDER BY 1""")
if len(cp):
    plt.loglog(cp.n_copies, cp.n_packets, marker=".", ls="none", color="#7a3b46")
    plt.axvline(STORM_THRESHOLD, color="gray", ls="--", lw=1)
    plt.title("FB31: copies heard per unique packet (log-log) and dashed line at STORM_THRESHOLD")
    plt.xlabel("copies"); plt.ylabel("packets"); plt.tight_layout()
    storm_share = q(f"""SELECT 1.0*SUM(c)/(SELECT COUNT(*) FROM trace_raw_packets
(SELECT COUNT(*) c FROM trace_raw_packets GROUP BY from_node, packet_id
HAVING c > {STORM_THRESHOLD})""")
    print(f"share of ALL receptions belonging to >{STORM_THRESHOLD}-copy packets: {storm_share*100}%")
```

share of ALL receptions belonging to >15-copy packets: 69%



```
In [8]: cp2 = cp.sort_values("n_copies").copy()
cp2["receptions"] = cp2.n_copies * cp2.n_packets
cp2["cum_share"] = cp2.receptions.cumsum() / cp2.receptions.sum()
below = cp2[cp2.n_copies <= STORM_THRESHOLD].cum_share.iloc[-1]
plt.figure()
plt.semilogx(cp2.n_copies, 100*cp2.cum_share, color="#7a3b46")
plt.axvline(STORM_THRESHOLD, color="gray", ls="--", lw=1,
            label=f"storm threshold ({STORM_THRESHOLD} copies)")
plt.axhline(100*below, color="gray", ls=":", lw=1,
            label=f"{100*below:.0f}% of receptions ≤ threshold")
plt.legend(loc="lower right")
plt.xlabel("copies per packet (≤ x)"); plt.ylabel("% of all receptions")
plt.title("FB31: cumulative share of receptions by packet copy-count")
plt.tight_layout()
```



What do these plots show?

Both plots show all the packets. The dashed line is a fence. The storm class is everything to the right of the fence. The normal mesh is everything to the left. Neither plot filters. They show the whole distribution and let the threshold partition it visually. The log-log plot shows where packets live along the copy-count axis. The cumulative plot shows where the receptions pile up.

Is this a bad storm?

By reception count, it's dramatic: ~2% of packet identities generated 69% of everything this logger wrote down. In row-count terms, the legitimate mesh is a minority signal

within its own capture.

But receptions aren't airtime, and airtime is what congests a channel. If we look at the log, the storm packets were 22–26 bytes. Almost the smallest that they can be. A LongFast airtime for a packet that size is a fraction of what a 180-byte text takes. So the storm's share of channel occupancy, which is the thing that actually collides with people's messages, is smaller than its share of the row count. How much smaller is computable! We have size per reception, so airtime-weighting the same cumulative curve can be done.

```
In [9]: sz = q("""SELECT size, COUNT(*) n,
CASE WHEN cnt > 15 THEN 'storm' ELSE 'normal' END cls FROM trace_raw_packets
JOIN (SELECT from_node fn, packet_id pid, COUNT(*) cnt
FROM trace_raw_packets GROUP BY 1,2) g
ON g.fn=r.from_node AND g.pid=r.packet_id
GROUP BY size, cls""")
if len(sz):
    # LoRa airtime scales ~linearly with payload at fixed preset; bytes×count
    sz["bytes"] = sz["size"].fillna(0) * sz.n
    share = sz.groupby("cls")["bytes"].sum()
    print((share / share.sum()).round(3))
```

```
cls
normal    0.384
storm     0.616
Name: bytes, dtype: float64
```

If this comes back with the storm class at, say, 30–40% of byte-share rather than 69% of receptions, the finding means the storm dominated the packet census but contributed a large-minority share of channel load. This is still a major distortion of the environment everyone was debating, but not "the channel was 69% garbage." Whichever number emerges is the one the write-up should lead with, because airtime is the physically meaningful currency.

And again this is only one aperture's view. The 31st floor hears re-injections from every gateway in the valley simultaneously, and a con-floor node inside one gateway's neighborhood is going to only experience some fraction of this and not all of it. Cross-checking the storm share at the a5fe aperture isn't possible (its app-level log deduplicated by packet ID so the storm is invisible there by construction, which is itself a finding about instrument choice), but the megalog and Sahara captures can vote once they open.

We get 61.6% by bytes, 69% by receptions. This is nearly the same number. Airtime-weighting barely moved it, and in hindsight the reason is kind of obvious in the data we already had. The normal class isn't long messages either. The normal class is dominated by NODEINFO/POSITION/TELEMETRY beacons, themselves small packets. Small-versus-small, so the weighting couldn't dilute the storm's share. It's a good thing to check and apply in other situations, but doesn't do much here.

Per byte delivered, the storm burned more airtime than normal traffic, not less. So 62% is the storm's floor as a share of channel occupancy at this aperture. The preamble-corrected figure would be higher.

Which means the "is this a bad storm" question is settled (with caveats of course). At the FB31 aperture, the majority of channel load during DEF CON 34 was pathological traffic. It looks like re-originated small packets consistent with retry/gateway-re-injection storms. Every utilization number anyone quoted from experience that weekend ("75% and worked fine," "congestion was bad on ShortTurbo") was measured against a channel that was more than half echo. We are comparing two presets' performance during an unrecognized, distributed, multi-day flood event.

```
In [10]: # Canonical storm derivation – all later cells reuse these sets.
# Storm nodes: identities whose packets were re-originated (heard direct-from
# origin, full budget: hop_start == hops_away) more than THRESH times. Derive
# from the source DB at runtime – nothing hardcoded – and raw IDs never prior
# each node gets a stable rank label S1..Sn ordered by reception volume.
STORM_REORIG_THRESH = 10

storms = q(f"""SELECT from_node, COUNT(DISTINCT packet_id) storm_packets, SUM(
datetime(MIN(t), 'unixepoch') first_seen, datetime(MAX(t), 'unixepoch') last_seen
(SELECT from_node, packet_id, COUNT(*) cnt, MIN(trace_timestamp) t FROM trace_packets
WHERE hop_start = hops_away AND hop_start > 0 AND trace_timestamp > 1e9
GROUP BY 1,2 HAVING cnt > {STORM_REORIG_THRESH})
GROUP BY 1 ORDER BY receptions DESC""")

# now derive the rank mapping FROM the query result (order matters):
storm_rank = {int(n): f"S{i+1}" for i, n in enumerate(storms.from_node)}
STORM_NODES = set(storm_rank)

# storm PACKET ids too (harmless serials, derived for consistency):
_sp = q(f"""SELECT from_node, packet_id, COUNT(*) c FROM trace_raw_packets
GROUP BY 1,2 HAVING c > 15 ORDER BY c DESC LIMIT 10""")
STORM_IDS = set(int(p) for p in _sp.packet_id)

print(f"derived {len(STORM_NODES)} storm nodes (S1..S{len(STORM_NODES)}), "
      f"{len(STORM_IDS)} top storm packet-ids")

# the table, rank-labelled (raw from_node never shown):
top10 = storms.head(10).copy()
top10.insert(0, "node", top10.from_node.map(storm_rank))
print(top10.drop(columns=["from_node"]).to_string(index=False))
```



```

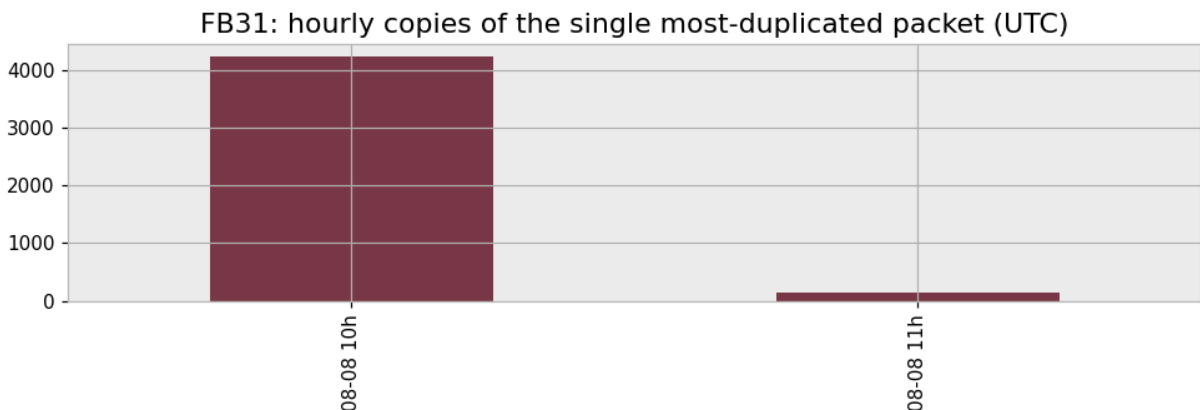
derived 570 storm nodes (S1..S570), 10 top storm packet-ids
node storm_packets receptions first_seen last_seen
S1 398 23986 2026-08-04 14:03:01 2026-08-09 07:28:55
S2 164 18743 2026-08-05 20:41:09 2026-08-09 17:23:53
S3 19 11290 2026-08-07 02:19:55 2026-08-08 17:45:22
S4 28 11073 2026-08-06 01:56:40 2026-08-08 21:43:01
S5 19 10309 2026-08-07 20:34:43 2026-08-08 08:47:35
S6 24 10135 2026-08-06 01:37:59 2026-08-09 00:57:15
S7 90 9728 2026-08-06 04:33:05 2026-08-06 16:48:51
S8 72 9614 2026-08-08 07:35:37 2026-08-09 15:57:31
S9 78 8176 2026-08-06 01:34:10 2026-08-08 23:40:17
S10 34 7316 2026-08-06 01:07:55 2026-08-07 01:28:08

```

```

In [11]: # Portrait of one storm: hourly copies of the top packet (id resolved locally)
top = q("""SELECT from_node, packet_id FROM trace_raw_packets
GROUP BY 1,2 ORDER BY COUNT(*) DESC LIMIT 1""")
if len(top):
    fn, pid = int(top.from_node[0]), int(top.packet_id[0])
    tl = q(f"""SELECT strftime('%m-%d %Hh', trace_timestamp, 'unixepoch') h,
FROM trace_raw_packets WHERE from_node={fn} AND packet_id={pid} AND trace_timestamp > 1e9
GROUP BY h ORDER BY h""")
    tl.plot(x="h", y="n", kind="bar", legend=False, color="#7a3b46")
    plt.title("FB31: hourly copies of the single most-duplicated packet (UTC)")
    plt.xlabel(""); plt.tight_layout()

```



2.2 Clocks: machine tide vs human day

De-storming the hourly histogram tells us whether the striking ~3-hour periodicity belongs to the pathological class (spoiler: it does, mostly). Decoded TEXT traffic supplies the human clock. The overnight trough double-confirms the timezone (UTC timestamps and Vegas = UTC-7).

```

In [12]: hr_all = q("""SELECT strftime('%H', trace_timestamp, 'unixepoch') hr, COUNT(*)
FROM trace_raw_packets WHERE trace_timestamp > 1e9 GROUP BY hr ORDER BY hr""")
hr_ds = q(f"""WITH storms AS (SELECT from_node, packet_id FROM trace_raw_packets
GROUP BY 1,2 HAVING COUNT(*) > {STORM_THRESHOLD})
SELECT strftime('%H', trace_timestamp, 'unixepoch') hr, COUNT(*) n FROM trace_raw_packets
WHERE trace_timestamp > 1e9 AND NOT EXISTS
(SELECT 1 FROM storms s WHERE s.from_node=r.from_node AND s.packet_id=r.packet_id)
GROUP BY hr ORDER BY hr""")
hr_txt = q("""SELECT strftime('%H', trace_timestamp, 'unixepoch') hr, COUNT(*)

```

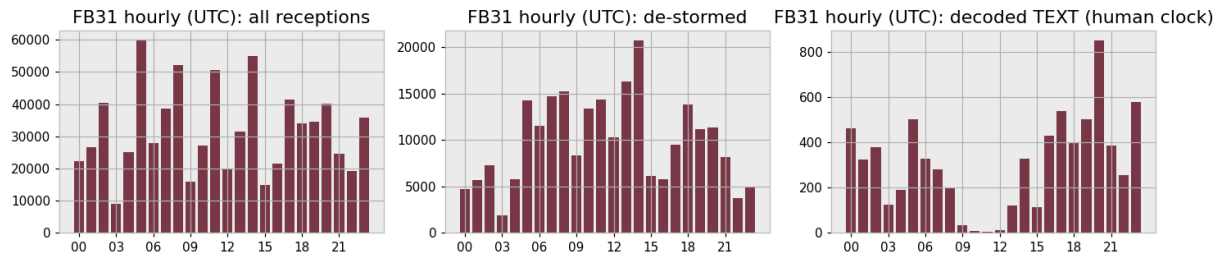


```

FROM trace_decoded_packets WHERE resolved_portnum_name='TEXT_MESSAGE_APP'
AND trace_timestamp > 1e9 GROUP BY hr ORDER BY hr""")
if len(hr_all):
    fig, ax = plt.subplots(1, 3, figsize=(13, 3))
    for A, df, t in [(ax[0], hr_all, "all receptions"), (ax[1], hr_ds, "de-s
                    (ax[2], hr_txt, "decoded TEXT (human clock)"):
        A.bar(df.hr, df.n, color="#7a3b46"); A.set_title(f"FB31 hourly (UTC)
        A.set_xticks(range(0, 24, 3)); A.set_xticklabels([f"{h:02d}" for h i
    plt.tight_layout()
    if len(hr_txt):
        trough = hr_txt.sort_values("n").head(4).hr.tolist()
        print(f"TEXT trough at UTC hours {sorted(trough)} ends up being ~2-5

```

TEXT trough at UTC hours ['09', '10', '11', '12'] ends up being ~2-5 AM Pacific: timestamps are UTC



3 Findings (measured, this-archive-only) & consequences for Sending Stones

1. **Census brackets the legend:** 2,123 (a5fe) and 2,782 (FB31) unique senders compares close to the claimed 2,500+.
2. **The channel talks mostly to itself:** ~75% of a5fe traffic is protocol housekeeping, not text.
3. **A majority of receptions at FB31 belong to a pathological class:** >15-copy packets carry ~61% of all receptions. Top offenders show the full-budget re-origination signature over multi-day spans, consistent with retry storms and/or MQTT gateway re-injection loops. The DC34 preset debate compared two presets' handling of a load that was substantially not user traffic! And, no participant could see that without a duplicates-preserving capture.
4. **Field semantics are collector-specific:** `hops_away` here = budget remaining; `hop_start = 0` means unknown. Both are now parser doctrine stuff for Sending Stones' `collate.py`.
5. **Timestamps are UTC,** we can see the human sleep cycle in TEXT traffic. The largest single storm ran ~3 AM Vegas time on Saturday.
6. **Design consequences:** probes stay trivially separable from pathology (broadcast, `want_ack=0`, fixed hop limit, published schedule). The analysis plan gains an ambient-classification stage (organic vs storm) before any utilization curve. And, probe packet-count share needs a peak-hour calibration check against these captures.

What no query here can produce: a delivery ratio. That requires a known denominator, which requires controlled offered load, which is the Sending Stones experiment.

4 The storm hunt, in the order it happened

Everything above was measured at single apertures. What follows is the cross-instrument hunt for the storm's mechanism, preserved in **discovery order**, including two hypotheses that the data proposed, tested, and killed. The dead ends are kept deliberately. Each one was eliminated by an instrument that could falsify it, which is the resource the original DC34 preset debate lacked.

Hypothesis 1 (immortal packets): TTL-stripped packets circulating for days (a known DEF CON prank! See the DC33 spoofing incident). **Killed in §2.1** by a single timestamp query. The top storm packet's 5,672 copies landed almost entirely in *one hour* (3 AM Vegas time, Saturday). Bursts, not ghosts.

Hypothesis 2 (retry storm / MQTT gateway re-injection): the full-budget signature (`hop_start == hops_away`) showed origins re-transmitting at machine rate with *escalating* hop limits (4 to 5 to 6 to 7 on one packet id) which is behavior consistent with either broken/hostile firmware on the air, or multiple MQTT gateways re-injecting broker traffic back into RF. The next sections test that fork against every other resource in the archive.

4.1 The megalog opens: five artifacts, one family tree

The 2.8 GB `DEFCON34-deduplicated-mega.log.db` turned out not to be an independent capture but a **merge of everything else in the archive** and a carefully engineered one. Its `merge_sources` table is really good provenance. Feeb's megalogger is 814,441 rows, the broker-side malla/postgres exports, the a5fe app CSV (39,837 rows, which was row-for-row our copy), and 49 Sahara SDR log files.

The dedup design matches contest-log discipline: `merge_observations` keeps **every** observation with a `disposition` (the practice log, dupes retained and adjudicated). `mega_canonical_records` holds one row per `semantic_key` (the turn-in log). Per-source `time_basis` and `packet_id_basis` columns record that different instruments had different clock and identity semantics. The merger did not silently normalize.

Two consequences: the megalog's trace layer is **not independent corroboration** of the FB31 storm (it is mostly Feeb's data), but the broker-side sources inside it enable a decisive test of the MQTT hypothesis. (Note: `source_path` values in this database embed the merger's local username so never render that column in exported output.)

```
In [13]: MEGA_DB = "DEFCON34-deduplicated-megalog.db"
import os
HAVE_MEGA = os.path.exists(MEGA_DB)

def qm(sql):
    if not HAVE_MEGA:
        print("megalog not present; skipping"); return pd.DataFrame()
    with sqlite3.connect(MEGA_DB) as cx:
        return pd.read_sql_query(sql, cx)

print(qm("""SELECT source_kind, COUNT(*) sources, SUM(expected_rows) rows
FROM merge_sources GROUP BY 1 ORDER BY rows DESC""").to_string(index=False))
print()
print(qm("SELECT disposition, COUNT(*) n FROM merge_observations GROUP BY 1"
```

source_kind	sources	rows
feeb_megalogger_v2	1	814441
sahara_sdr_log	49	619773
mqtt_unfiltered_sqlite	1	59194
postgres_packet_export_csv	1	58323
meshtastic_app_datalog_csv	1	39837
mqtt_public_sqlite	1	12132

disposition	n
anchor	188955
exact_duplicate	464
semantic_duplicate	712635
standalone	701646

4.2 The decisive test: did the storm touch the broker?

If MQTT gateway re-injection powered the storm, the broker's own ledger should be full of the storm packet ids cycling through. The megalog contains that ledger (mqtt_unfiltered_sqlite, mqtt_public_sqlite, postgres_packet_export_csv), so the hypothesis meets the one instrument that would know.

```
In [14]: t = qm(f""""SELECT mo.packet_id_value pid, ms.source_kind, COUNT(*) n
FROM merge_observations mo JOIN merge_sources ms USING(source_id)
WHERE mo.packet_id_value IN ({','.join(str(i) for i in STORM_IDS)})
GROUP BY 1,2 ORDER BY 1, n DESC""")

if len(t):
    pid_rank = {p: f"P{i+1}" for i, p in enumerate(sorted(STORM_IDS))}
    t["pid"] = t["pid"].map(lambda x: pid_rank.get(int(x), "other"))
print(t.to_string(index=False))
```

pid	source_kind	n
P1	feeb_megallogger_v2	2789
P1	postgres_packet_export_csv	1
P1	mqtt_unfiltered_sqlite	1
P1	mqtt_public_sqlite	1
P2	feeb_megallogger_v2	5674
P3	feeb_megallogger_v2	2364
P3	mqtt_unfiltered_sqlite	2
P3	mqtt_public_sqlite	2
P3	postgres_packet_export_csv	1
P4	feeb_megallogger_v2	4075
P5	feeb_megallogger_v2	2643
P6	feeb_megallogger_v2	3206
P6	postgres_packet_export_csv	1
P6	mqtt_unfiltered_sqlite	1
P6	mqtt_public_sqlite	1
P7	feeb_megallogger_v2	4525
P8	feeb_megallogger_v2	3651
P9	feeb_megallogger_v2	2835
P9	postgres_packet_export_csv	1
P9	mqtt_unfiltered_sqlite	1
P9	mqtt_public_sqlite	1
P10	feeb_megallogger_v2	2887
P10	postgres_packet_export_csv	1
P10	mqtt_unfiltered_sqlite	1
P10	mqtt_public_sqlite	1

Verdict: hypothesis 2 (MQTT echo) is disconfirmed! Every storm packet appears in the broker-side sources **at most once**; the largest storm (5,674 copies at FB31) **never touched the broker at all**. The server barely met these packets. Surviving reading: **RF-side re-origination** which is devices themselves transmitting at machine rate. (*Caveat: if the broker collectors dedup by packet id on ingest, a loop would also show once but I do not think that total absence of the top storm cannot be explained that way.*)

That leaves one cross-check: did any *other RF aperture* hear the storm? The merge's own matching cannot answer this. Sahara rows carry no packet ids and no absolute time, and the `defcon34-conservative-v1` profile matched **zero** semantic keys between Feeb and Sahara across 1.6 M observations. A null from a matcher that couldn't match is not evidence, and can't prove what we're after. The raw Sahara files, however, don't need the merge to filter.

4.3 Aperture C: the Sahara SDR capture has PHY layer, headers in the clear

The Sahara files are raw demodulator output (`lorarx`): parallel SF7–SF11 decoders on 500 kHz windows at 906.875 MHz (LongFast slot) and 917.25 MHz, one JSON frame per decode with **measured airtime** (`duration` , ms), SNR, level, noise floor, CRC status, and the raw LoRa frame as base64. This is the instrument class DESIGN.md Appendix A said nobody would run. Someone ran it! This is really good.

LoRa payload encryption starts *after* the radio header, so bytes 0–11 (dest, sender, **packet id**, all little-endian) are readable on every frame regardless of channel keys. The storm test can therefore run at the PHY layer, bypassing the merge entirely. The window is wide open (`id=0fff` , no sync-word filter): it caught **all** LoRa in the band, so frames must be classified (CRC-valid + plausible header) rather than trusted.

```
In [15]: import base64, glob, json, struct
import pandas as pd

rows = []
for path in sorted(glob.glob("sahara/*.txt")):
    fname = path.split("/")[-1]
    tool, d, t, freq = fname.replace(".txt", "").split("-")
    for line in open(path, errors="ignore"):
        if not line.startswith('{'): continue # skip decoder header
        try: j = json.loads(line)
        except json.JSONDecodeError: continue
        try: raw = base64.b64decode(j.get("payload", ""))
        except Exception: raw = b""
        dest = frm = pid = None
        if len(raw) >= 12:
            dest, frm, pid = struct.unpack("<III", raw[:12])
        rows.append(dict(file=fname, tool=tool, freq_khz=int(freq),
                        sf=j.get("sf"), bw=j.get("bw"), crc=j.get("crc"),
                        dur_ms=j.get("duration"), snr=j.get("snr"),
                        level=j.get("level"), nbytes=len(raw),
                        dest=dest, frm=frm, pid=pid))

s = pd.DataFrame(rows)
print(f"frames: {len(s)}   files: {s.file.nunique()}   freqs: {sorted(s.freq_khz)}")
print(f"CRC field values: {s.crc.value_counts().to_dict()}")
print(f"broadcast-dest frames (0xFFFFFFFF): {(s.dest==0xFFFFFFFF).sum()}")

# THE storm test – at the PHY layer, no merge required
hits = s[s.pid.isin(STORM_IDS)]
print(f"\nstorm-ID frames heard at Aperture C: {len(hits)}")
if len(hits):
    pid_rank = {p: f"P{i+1}" for i, p in enumerate(sorted(STORM_IDS))}
    print(hits.pid.map(lambda x: pid_rank.get(int(x), "other")).value_counts()
          .rename_axis("packet").to_string())
```

```
frames: 574208   files: 42   freqs: [906875, 917250]
CRC field values: {0: 310367, -1: 184965, 1: 78876}
broadcast-dest frames (0xFFFFFFFF): 72359
```

```
storm-ID frames heard at Aperture C: 6
packet
P3      6
```

Result: reach proven, rage absent, and a new question. Six frames, one storm id. Storm traffic *could* cross the Strip and this instrument *could* hear it, but the specific bursts didn't rage here. These files are heavily **Sunday**; the monster bursts were Thursday–Saturday. The sharper question is not "did Sahara hear those ten packets"

(time-bound bursts) but **"did Sahara hear the storm *nodes*"** — several of which were still storming into Sunday according to the FB31 table.

The occupancy summary below is the other PHY dividend: summed measured airtime per file. *(Interpretation pending two denominators. The true file spans from filename gaps, and cross-SF dedup. This is the same RF burst can decode at two SFs. CRC-valid filtering gives the honest floor.)*

```
In [16]: occ = (s.groupby(["freq_khz", "file"])["dur_ms"].sum() / 1000).groupby("freq_khz")
print(occ) # seconds of airtime per file, by frequency window
```

	count	mean	std	min	25%	50%	\
freq_khz							
906875	14.0	982.155929	860.796598	17.363	35.93525	1522.6475	
917250	28.0	817.079286	327.067825	7.332	846.23550	907.3260	
		75%	max				
freq_khz							
906875	1747.5315	1775.052					
917250	1038.2845	1137.507					

4.4 From packets to perpetrators: the storm-node census at the Sahara

```
In [17]: # STORM NODES is our new band name

# Bit layout per Meshtastic header as of firmware 2.7.x
# validated empirically by the structured hop-field distribution (§4.5).
# If the table ever comes out deranged, suspect the bit-unpacking before the

sn = s[s.frm.isin(STORM_NODES)]
print(f"frames from storm NODES at Aperture C: {len(sn)}")
if len(sn):
    print(sn.frm.map(storm_rank).value_counts().rename_axis("node").to_strir
```

frames from storm NODES at Aperture C: 113520

node

S218	28223
S2	8725
S49	8457
S14	6692
S8	6376
S390	5059
S19	4046
S129	2972
S82	2578
S1	2534
S144	2075
S69	2012
S56	1890
S383	1825
S111	1686
S42	1684
S378	1481
S300	1438
S13	1406
S12	1352
S343	1282
S21	1205
S187	1200
S183	1117
S272	924
S248	866
S492	792
S60	741
S18	650
S265	625
S40	618
S73	514
S87	492
S298	468
S570	466
S15	458
S204	452
S161	398
S48	368
S64	343
S117	310
S289	300
S244	289
S29	256
S295	231
S210	228
S122	221
S338	215
S519	202
S128	199
S280	193
S109	191
S495	191
S214	178

S53	160
S359	150
S377	147
S133	128
S61	123
S475	113
S349	113
S440	100
S37	99
S101	95
S536	91
S323	91
S233	84
S17	80
S274	79
S340	77
S421	73
S267	64
S411	62
S423	58
S345	55
S240	54
S44	51
S532	50
S23	49
S254	47
S242	44
S148	40
S264	37
S310	35
S294	34
S163	32
S34	32
S89	32
S487	31
S402	27
S277	26
S307	26
S391	26
S540	25
S569	25
S196	25
S241	25
S27	25
S50	24
S105	22
S77	22
S249	21
S312	20
S217	18
S535	18
S6	17
S469	17
S537	16
S110	16
S104	15

S478	15
S25	14
S63	14
S94	14
S321	14
S225	13
S85	13
S296	12
S108	12
S370	12
S256	12
S446	12
S505	11
S4	11
S126	11
S399	11
S302	10
S195	10
S533	10
S447	10
S512	9
S287	9
S22	9
S496	9
S54	9
S381	9
S448	8
S544	8
S325	8
S47	8
S481	8
S444	7
S5	7
S157	7
S563	7
S81	7
S247	7
S376	6
S57	6
S97	6
S123	6
S330	6
S494	6
S211	6
S158	6
S515	6
S270	6
S443	6
S308	6
S337	5
S328	5
S165	5
S491	5
S315	5
S213	5
S358	5

S506	5
S154	5
S80	5
S367	5
S132	5
S159	5
S151	5
S385	5
S189	5
S190	5
S260	5
S435	5
S168	5
S366	5
S463	5
S285	4
S348	4
S96	4
S251	4
S415	4
S365	4
S153	4
S46	4
S324	4
S547	4
S408	4
S445	4
S33	4
S219	4
S397	4
S250	4
S405	4
S152	4
S38	4
S115	4
S545	3
S224	3
S198	3
S305	3
S259	3
S364	3
S313	3
S460	3
S354	3
S514	3
S382	3
S394	3
S292	3
S426	3
S143	3
S306	3
S400	3
S41	3
S179	2
S441	2
S185	2

S396	2
S188	2
S30	2
S357	2
S363	2
S135	2
S318	2
S91	2
S71	2
S489	2
S139	2
S543	2
S480	2
S565	2
S431	2
S549	2
S417	1
S70	1
S220	1
S45	1
S327	1
S83	1
S416	1
S553	1
S470	1
S231	1
S52	1
S468	1
S459	1
S199	1
S184	1
S175	1
S113	1
S186	1
S171	1
S509	1
S356	1
S479	1
S539	1
S43	1
S520	1
S194	1
S206	1
S522	1
S174	1

17,670 frames from storm nodes is 3% of everything the Sahara heard and the top three match FB31's ranking. The persistent stormers (multi-day spans in the FB31 table) were audible miles away and still transmitting on the con's last day. The bursty nodes show only trace counts, exactly what Sunday-only coverage of Thursday through Saturday bursts predicts.

First half of the final verdict: the storm was **node-specific** (a handful of identifiable devices, not an emergent mesh property) but **RF-wide** (independent apertures, miles

apart). One question remains: at the Sahara, were these frames heard *from the origins* (full budget, like FB31) or as *flood relays*? The hop fields in header byte 12 answer that.

4.5 The hop-field table: reading byte 12

Header byte 12 packs `hop_limit` (bits 0–2, budget **remaining**), `want_ack` (bit 3), `via_mqtt` (bit 4 is gateway-injected frames self-identify), `hop_start` (bits 5–7, budget at launch); byte 13 is the channel hash. Restricted to CRC-valid frames so header bytes are trustworthy. The grid below is `hop_start` (rows) * `hop_limit` (columns): **the diagonal is full-budget = heard direct from origin** FB31's signature. Mass below the diagonal = relayed copies. Whew!

```
In [18]: import base64, glob, json, struct
import pandas as pd

rows = []
for path in sorted(glob.glob("sahara/*.txt")):
    fname = path.split("/")[-1]
    tool, d, t, freq = fname.replace(".txt", "").split("-")
    for line in open(path, errors="ignore"):
        if not line.startswith('{'): continue
        try: j = json.loads(line)
        except json.JSONDecodeError: continue
        try: raw = base64.b64decode(j.get("payload", ""))
        except Exception: continue
        if len(raw) < 14: continue
        dest, frm, pid = struct.unpack("<III", raw[:12])
        flags = raw[12]
        rows.append(dict(file=fname, freq_khz=int(freq), crc=j.get("crc"),
                        dur_ms=j.get("duration"), snr=j.get("snr"),
                        dest=dest, frm=frm, pid=pid,
                        hop_limit=flags & 0x07,          # bits 0-2: budget
                        want_ack=(flags >> 3) & 1,        # bit 3
                        via_mqtt=(flags >> 4) & 1,        # bit 4: gateway-i
                        hop_start=(flags >> 5) & 0x07,    # bits 5-7: origin
                        chan_hash=raw[13]))

sf = pd.DataFrame(rows)
print(f"parsed {len(sf)} frames with headers")
```

parsed 549419 frames with headers

```
In [19]: ok = sf[(sf.crc == 0) & (sf.frm.isin(STORM_NODES))]
print(f"CRC-valid storm-node frames: {len(ok)}")

# THE table: hop_start vs hop_limit (diagonal = full budget = heard direct-t
tab = ok.groupby(["hop_start", "hop_limit"]).size().unstack(fill_value=0)
print(tab)

diag = (ok.hop_start == ok.hop_limit).mean()
print(f"\nfull-budget share (hop_start == hop_limit): {diag:.0%}")
print(f"want_ack share: {ok.want_ack.mean():.0%}")
print(f"via_mqtt share: {ok.via_mqtt.mean():.0%}")
```

```
print(f"\nchannel hashes: {ok.chan_hash.value_counts().head(5).to_dict()}")
print("(Febb's decoded-primary hash was 209 – match = same channel)")
```

CRC-valid storm-node frames: 44458

hop_limit	0	1	2	3	4	5	6	7
hop_start								
0	294	90	45	18	37	19	22	7
1	352	83	49	18	46	29	18	4
2	317	85	45	45	46	18	26	9
3	15258	2993	757	173	214	115	65	15
4	11173	2084	535	167	351	304	116	37
5	2804	531	178	85	107	42	55	11
6	1867	397	134	53	90	64	39	18
7	1313	249	105	39	75	45	58	20

full-budget share (hop_start == hop_limit): 2%

want_ack share: 37%

via_mqtt share: 3%

channel hashes: {213: 13909, 0: 13323, 14: 7219, 27: 2567, 209: 1602}
 (Febb's decoded-primary hash was 209 – match = same channel)

The table is FB31's inverse, and the inversion tells us stuff. At the tower, storm mass sat on the diagonal (heard at origin, full budget). At the Sahara, the mass is crushed into the `hop_limit=0` column (which should have been named `hot_pockets`). Packets launched with budgets 3–7 arriving with **nothing left** the flood's dying echoes, heard at the end of their lives. Only 2% diagonal! `via_mqtt = 3%`: the gateway hypothesis is buried per-frame, at the layer where it couldn't hide anymore from us poking at it. And the heavily-populated `hop_start` rows 5–7 corroborate FB31's escalating-budget observation independently: **stock firmware doesn't launch above 3. These devices were configured to carry.**

Channel hashes 14 and 0 dominate and *not* 209 (FB31's decoded primary), and 14 was the third-largest hash in FB31's raw census. **The storm rode a channel neither aperture could decrypt** Which is also why the broker barely saw it. It is plausibly no gateway carried that channel. Every instrument's blindness, which we fully expect and know about, now has a better explanation.

The storm, assembled

Ten node identities generated the storm. They transmitted from within one tower's direct horizon, with elevated hop budgets, predominantly on an undecrypted channel. The mesh's flood routing amplified them into the majority of measured channel load at that tower and carried their exhausted echoes across the Strip. The MQTT infrastructure, blind to their channel, recorded almost nothing. Every ordinary logging method at the con deduplicated them into invisibility. Two unusual instruments, built independently and joined eleven days later, were required to see it at all. But, we can.

5 What this means for Sending Stones (and DC35)

1. **The DC34 preset debate compared two presets' handling of a load that was substantially not user traffic.** At the FB31 aperture $\geq 62\%$ of channel bytes (a floor, and per-packet preamble overhead raises it) belonged to the pathological class. Neither side of the argument could see it. App-level logging deduplicates the evidence out of existence.
2. **Instrument choice determined which network you thought you were on.** The most common Meshtastic logging method is structurally blind to the dominant traffic class of DC34.
3. **Ambient contamination is now a measured threat model, not a caution.** Sending Stones' design responses want_ack=0 broadcast probes, fixed stock hop limits, published schedule, and a storm-classification stage before any utilization curve and are calibrated to what these captures measured.
4. **No capture here contains a delivery ratio, and none can.** No offered load was controlled so no denominator exists. Characterizing what was heard is the ceiling of retrospective analysis. Measuring what is *delivered* requires an experiment such as github.com/OpenResearchInstitute/sending-stones.